Open in app ↗ Sign up Sign in

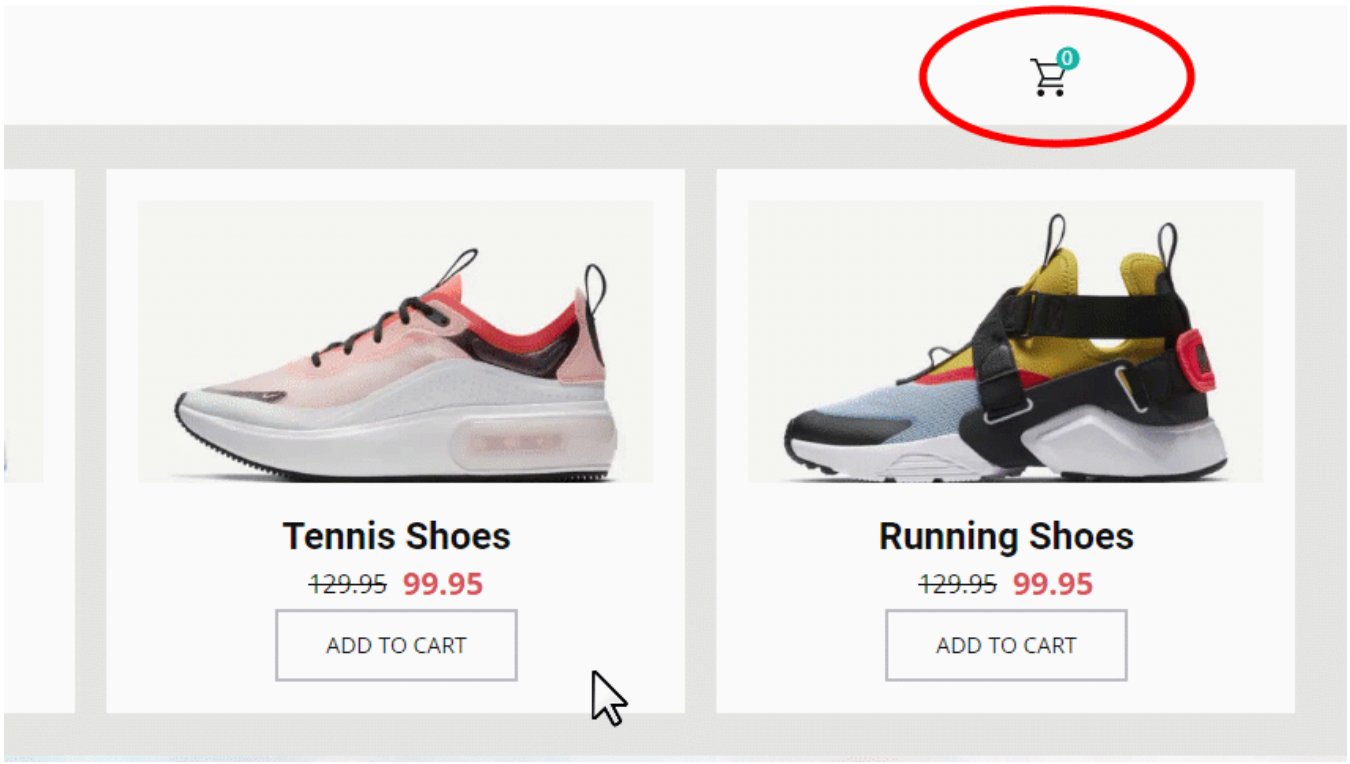
5 Different Techniques for Cross Micro Frontend Communication

Vishal Sharma · Follow
6 min read · Jun 12, 2022

Listen Share

The title itself signifies what we are going to discuss in this article. But before we even start, let's try to understand why cross-micro frontend communication should even be a point of discussion.

Consider the example of an e-commerce site:



As we can see, whenever a user is adding a product to the cart, the count of added products is increasing on the cart icon.

Following the concept of domain boundaries to split the app into multiple micro frontends, we can think of two different micro frontends here:

A Catalog and A Cart.

In the monolith world, each domain sits inside the same App which makes it easier to pass the data from one to the other domain. Different frameworks provide their way to pass data between these domain boundaries. One of the most common ways is to maintain the data at a higher layer and pass that to different boundaries via props.

Now, this is not as straightforward when the whole app is divided into multiple smaller apps running independently and embedded in a single container app aka Micro Frontends. So the architecture with its whole list of advantages comes up with a set of complexities also and one of those is Cross micro frontend communication.

In this article, we are going to discuss five different techniques for micro frontend communications and the pros and cons of each one.

> Please note that *t*he code snippets added in the article are for example purpose only and might need some tweaks before actually running.

1. Using Props

This is the most basic technique for cross micro frontend communication where the container app is maintaining the state and passing it further to the required micro frontends.

Taking the example of the e-commerce site again, the Catalog micro frontend is taking a callback to add a product to the cart which is responsible to increase the total count and passing the count further to the Cart micro frontend.

> *This technique is effective in case of build time micro frontend integration pattern as the application needs to render the micro frontends as components only and pass the respective props to those.*

Pros

- One of the very well-known techniques of data passing in component-based architecture.
- Most of the frameworks support this way.
- One can always use framework structs to avoid prop drilling issues e.g. React Context etc.

Cons

- Adds a lot of coupling between the micro frontends and the container app.
- Hard to achieve if two micro frontends are not using the same framework
- It can impact the overall performance of the application as multiple unwanted layers will be re-rendered with every state change.

2. Using Platform Storage Apis

In this technique, we can leverage the platform's built-in storage APIs like Local Storage in browsers and Async Storage in cross-platform solutions like react native for mobile micro frontends also.

We can create a simple storage utility library exposing setter and getter from storage APIs. Now, instead of making micro frontends communicate via container app, each different micro frontend can set as well as read the data directly using the utility.

In the example below, Catalog and Cart, both the micro frontends are consuming the storage utility library. The catalog micro frontend is setting the latest count of added products in the storage which is further read by the Cart micro frontend.

> *This technique can work for build as well as run time integration of micro frontends. Since the storage apis provided by different platforms do not follow the subscription pattern, the micro frontend will not immediately read the new data when it is updated by the other micro frontend. So an asynchronous and pub sub model needs to be built. This technique will be more useful when two micro frontends are rendered in two different screens as the reader can fetch the updated data on mount itself.*

Pros

- Available for browsers as well as mobile devices. Local storage for browsers and Async storage for mobile apps.
- Less coupling compares to passing props between the App and micro frontend but hard to debug which micro frontend is setting the data.

Cons

- Not a scalable solution for bigger applications. But can be used for a small set of data. It is always good to namespace the data set into platform storage according to the app name to avoid ambiguity.
- Not a secure technique for saving protected data.

3. Using Custom Events

This technique is more suitable for web micro frontends and a more scalable technique for run-time micro frontends. The main idea here is to utilize the browser-inbuilt custom events APIs to publish the events with the data from one micro frontend and the other micro frontends subscribe to the events to get the data. This technique is closest to the events-driven architecture in the microservices world.

In the example below, Catalog and Cart, both the micro frontends are consuming the event library. Catalog micro frontend is creating an event `ADD_TO_CART` in the event registry and then publishing the same. Cart micro frontend is subscribing to the same event and is getting the count of updated products in the cart via the event data. We also need to make sure that the subscribed events get unsubscribed once the component is unmounted.

> *This technique can also work for build time as well as run time integration of micro frontends and can work better if the cross communication is within the same page as events need to be subscribed before publishing.*

Pros

- The inbuilt solution in-browser platform.
- Very much close to asynchronous event-based architecture in the microservices world. Easy for the backend developers to understand as well.
- High setup cost but easy to scale.
- Build a generic mechanism that all the micro frontend teams can follow.

Cons

- Not achievable in the case of mobile micro frontends

4. Using a custom Message bus

This technique is almost similar to the above one but instead of relying on the browser custom events API, we can build our pub-sub mechanism.

The message bus library can expose methods to publish, subscribe and unsubscribe the events. The publish event needs to make sure that all the subscribed handlers are invoked once the event is published.

In the example below, Catalog and Cart, both the micro frontends are consuming the message bus library instance passed via the container app. Catalog micro frontend can publish an event `ADD_TO_CART` which Cart micro frontend is subscribed to using the same message bus.

> *This technique can be more useful for build time integration in case of mobile micro frontends as there is no concept of custom events there. It is very important to make sure that all the micro frontends are using the same instance of the message bus as an application should have only one registry for all the events published.*

Pros

- A custom-made solution equivalent to message bus in microservices implementation.
- High setup cost but easy to scale.
- Libraries like Postal.js are available in the market.

Cons

- Similar to the custom events technique, it can be hard to make all the micro frontends teams follow the same pattern.

5. Using Post Message passing in iFrames

This technique is only relevant if you are building runtime micro frontends using iFrames.I feel that there is a better integration pattern for building micro frontends than using iFrames but this technique can be very much useful when you need to inject a micro frontend inside another running app. One can always read the official docs to understand the pros and cons of the whole technique.

. . .

In the end, the usage of a particular mechanism would mainly depend upon the micro frontend integration pattern (build time or run time) but also on the state of the project as well. e.g. One might not want to set the whole message bus system for an MVP as it has a huge setup cost with it. So the idea should always be to start simple and build on top of that as the project grows.

Micro Frontends | Cross Mfe Communication

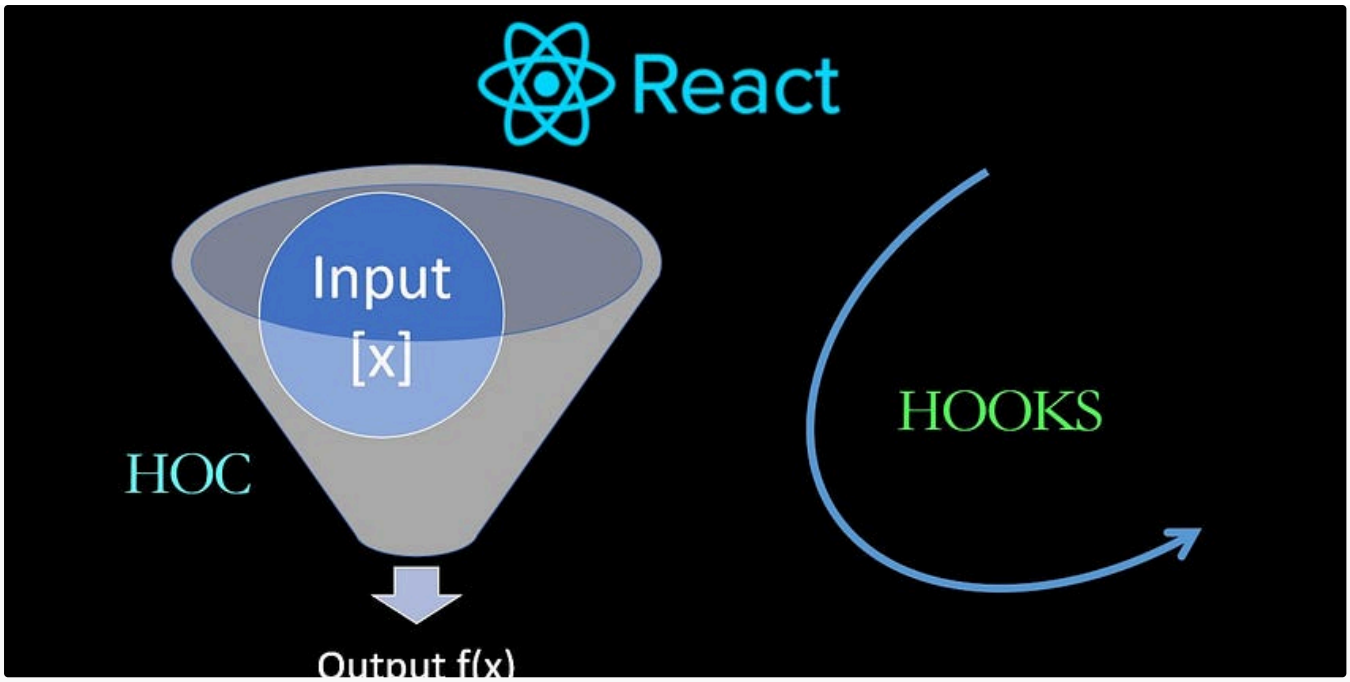
Follow

Written by Vishal Sharma

129 Followers

Front End Engineer at /thoughtworks.

More from Vishal Sharma

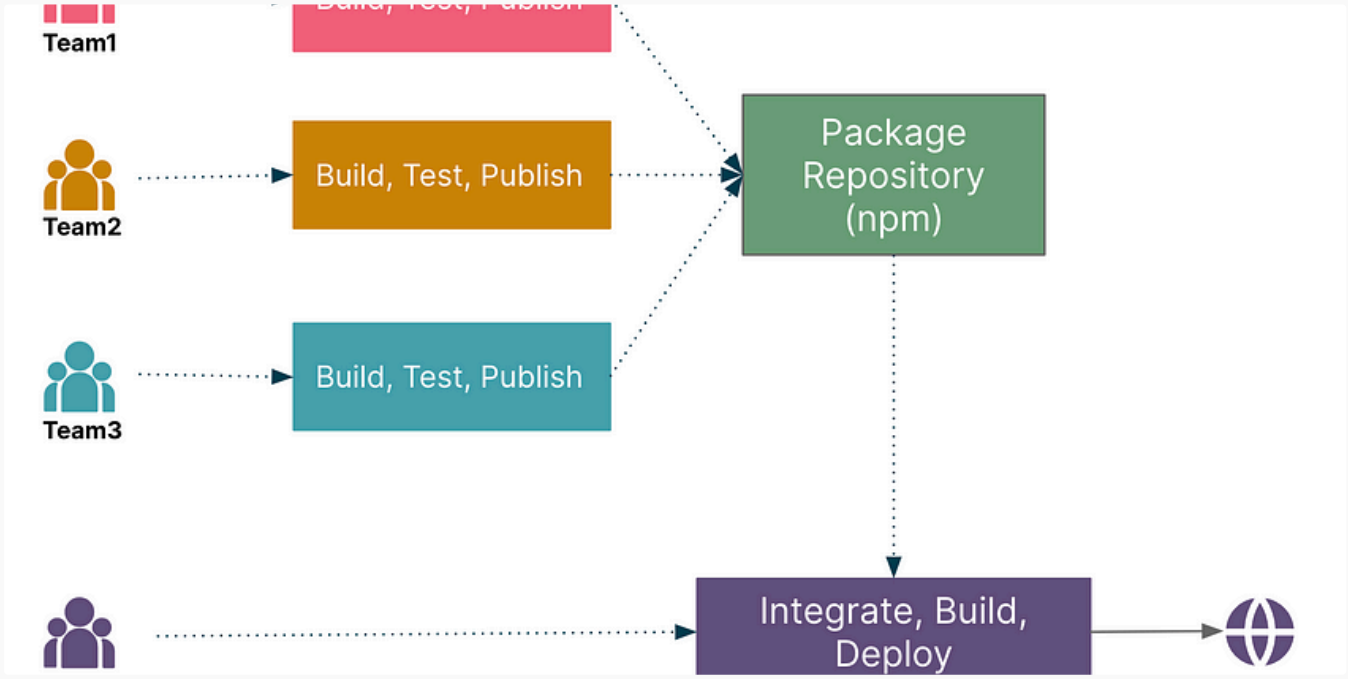


Vishal Sharma

Following DRY principle for Api Calls in React

In most of the React projects, we have components where we need to fetch the data from an api on mount. React provides us lifecycle...

Nov 5, 2021 296 2



Vishal Sharma

How to Compose Micro Frontends at Build Time

We have been building npm packages and using those inside the frontend application for eternity now. Build time composition of Micro...

Aug 5, 2022 32 1

Vishal Sharma

Setting up Typescript + EsLint + Jest Project

A lot of node projects have a setup which usually include Typescript for static type checking, esLint for static code analysis and Jest as...

Jul 7, 2021 51 2

Vishal Sharma

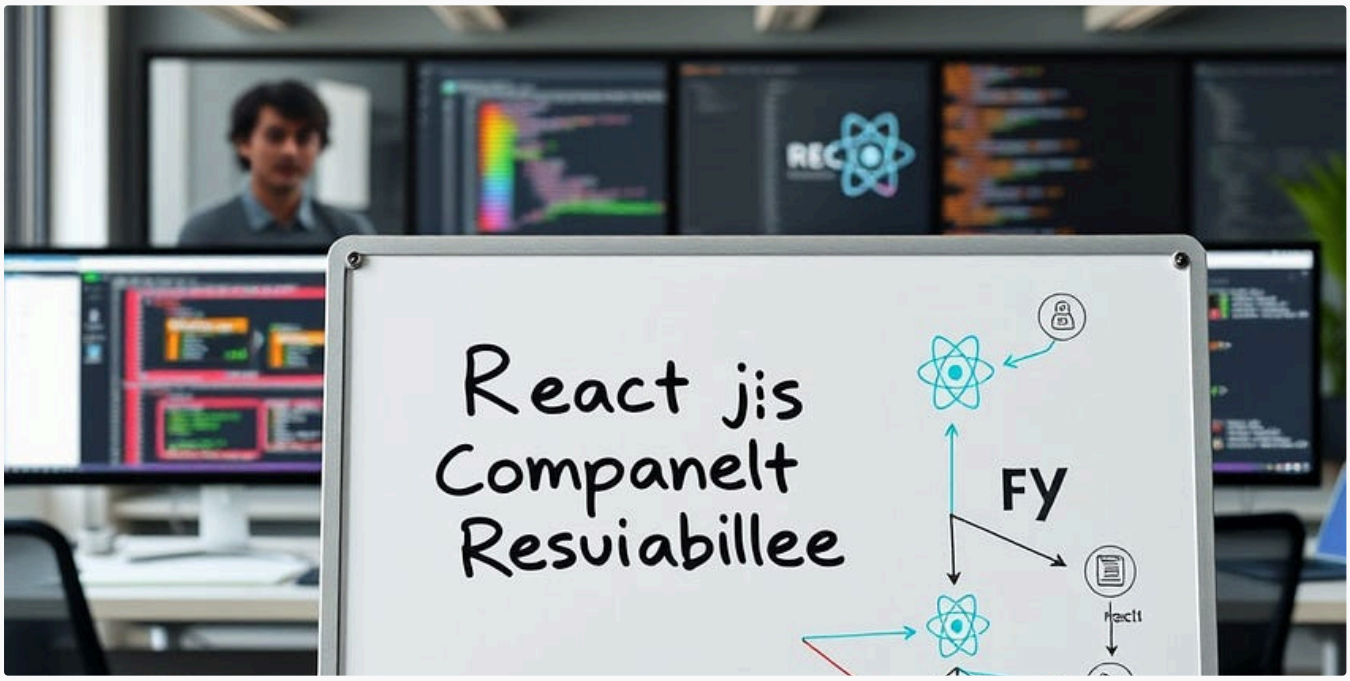
SSR and Web Components— A Contentious Couple

Usually, we talk about the technologies that work best together. This article is the exact opposite where I would like to discuss the...

Jul 31 9

See all from Vishal Sharma

Recommended from Medium

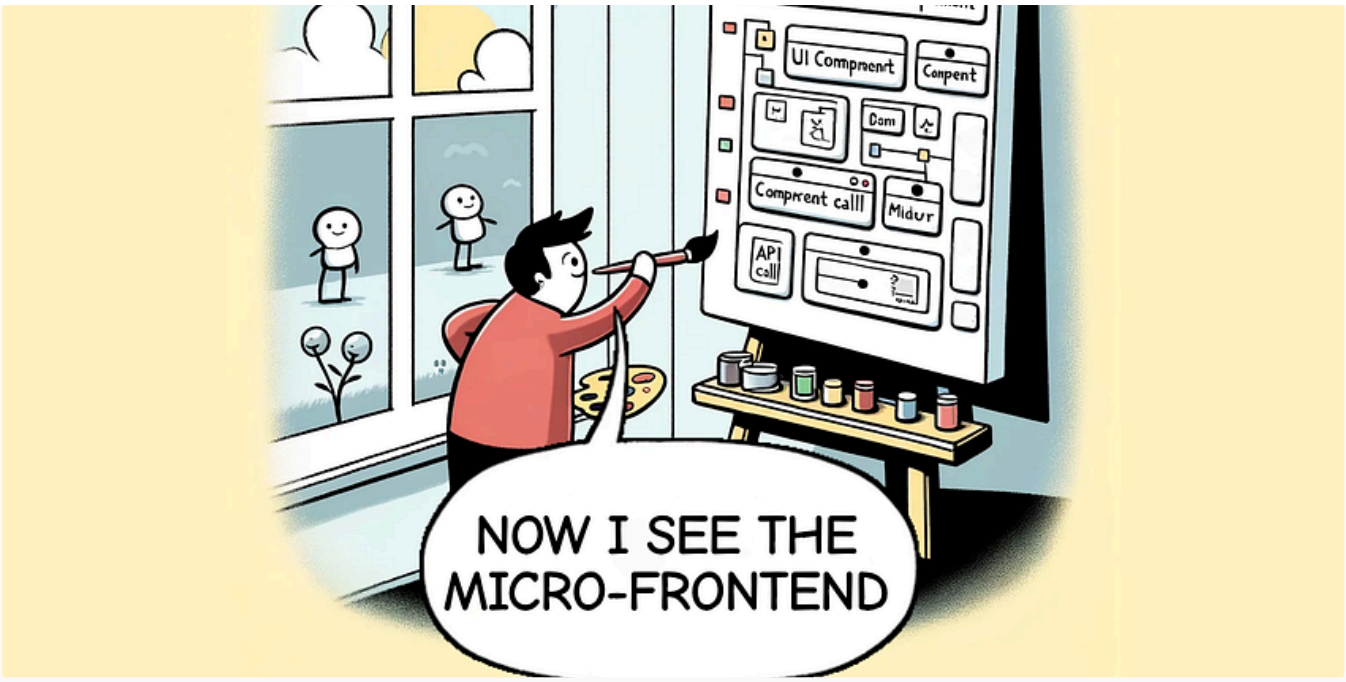


Asian Digital Hub

React Component Reusability in Micro-Frontends

Are you tired of reinventing the wheel every time you build a new micro-frontend? Do you find yourself copying and pasting React components...

Sep 28



David Rodenas PhD

Overcoming the Implementation Challenges of Micro-Frontends

A comprehensive guide of the micro-frontend architecture and implementation challenges.

Mar 30 137 3

Lists

Staff Picks

750 stories · 1372 saves

Stories to Help You Level-Up at Work

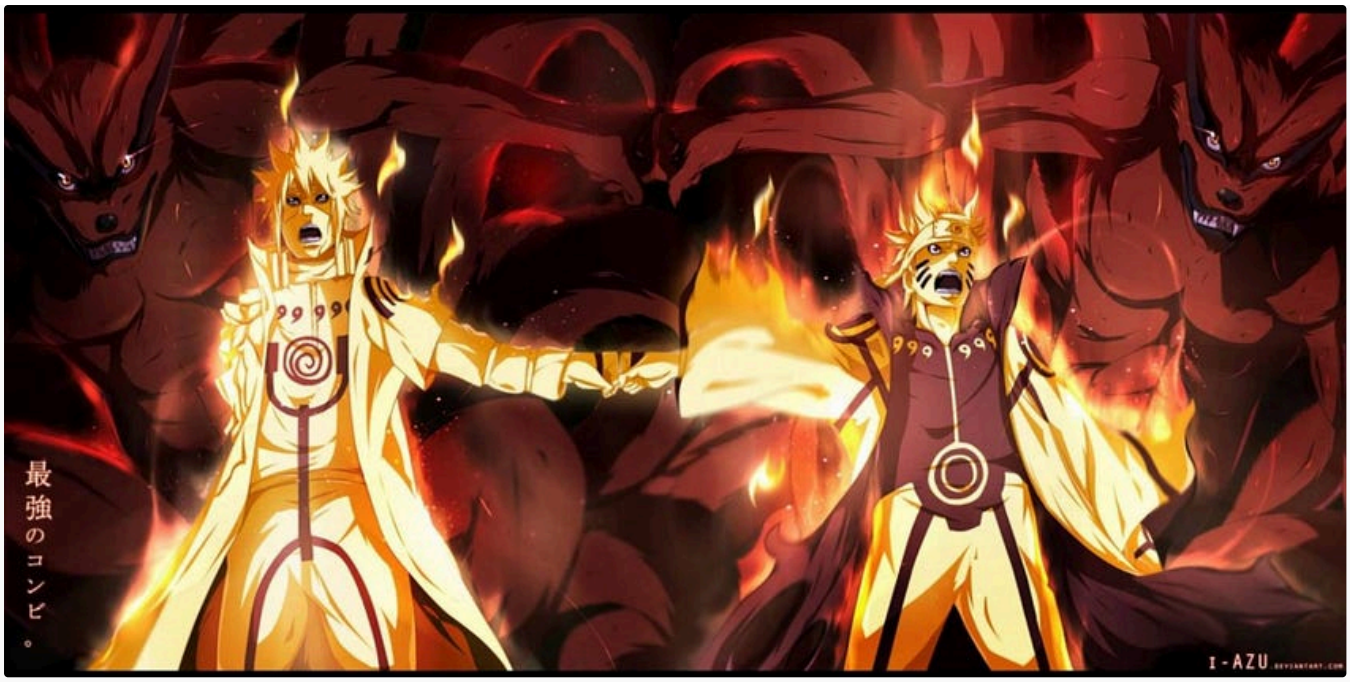
19 stories · 836 saves

Self-Improvement 101

20 stories · 2869 saves

Productivity 101

20 stories · 2443 saves

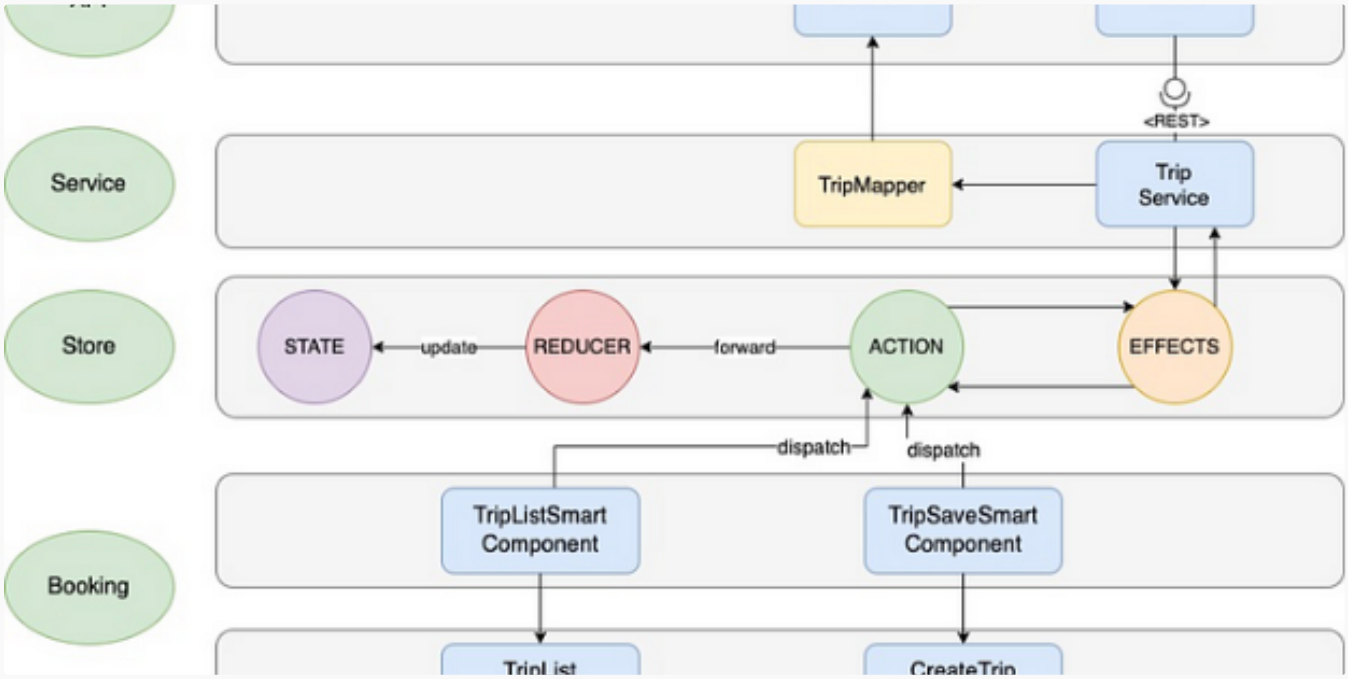


Guhaprasaanth Nandagopal

React vs. Next.js: Microfrontend Matchup—Making Them Play Nice with Vite

Microfrontend (MFE) architecture allows you to build complex applications by composing multiple smaller frontend applications, each...

Jun 14 26

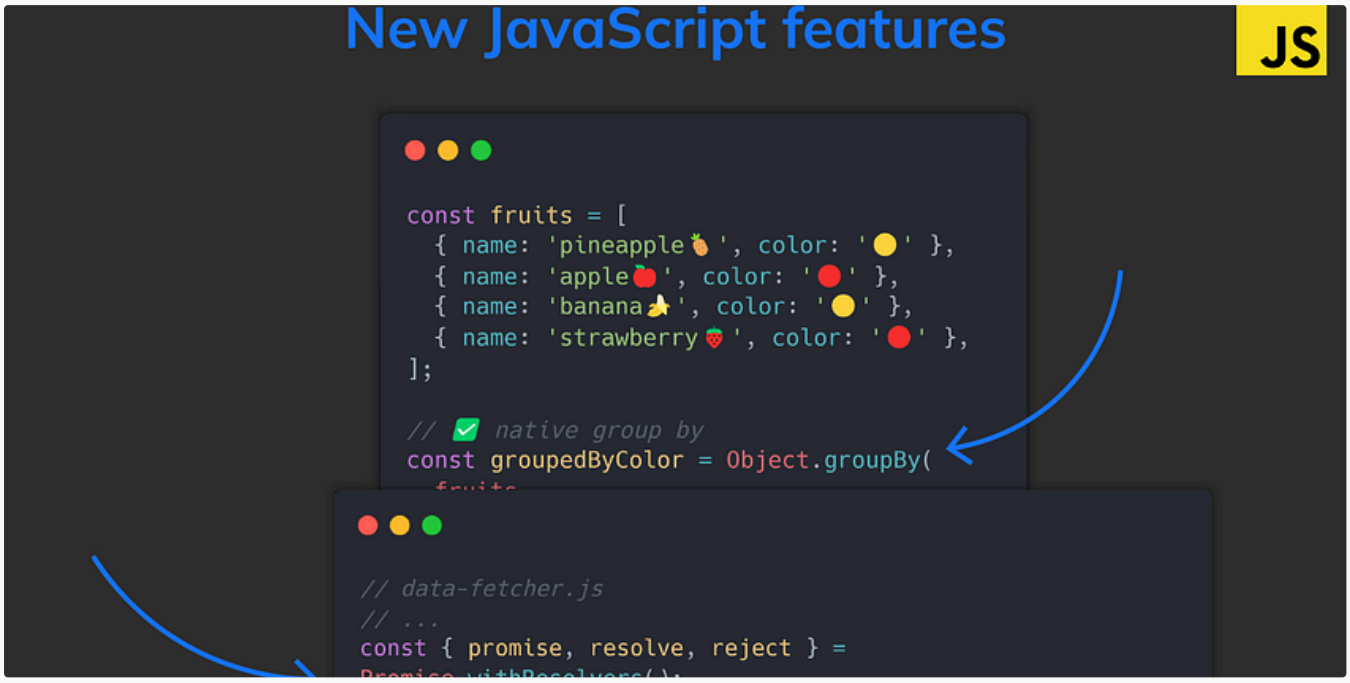


J.D Christie

Building a Clean and Scalable Frontend Architecture

Digital and technological development continues unstoppable, so, if we talk about the design of web applications, we cannot lose sight of...

Jan 13 1.1K 7



Tari Ibaba in Coding Beauty

5 amazing new JavaScript features in ES15 (2024)

5 juicy ES15 features with new functionality for cleaner and shorter JavaScript code in 2024.

Jun 2 3K 20

Arunangshu Das

Handling Routing in a Microfrontend Architecture

As modern web applications grow in complexity, the monolithic approach can become a bottleneck for development speed, scalability, and...

Jun 2 6

See more recommendations